

Otimização de hiperparâmetros de redes MLP para correção temporal de valores de PM2.5 utilizando busca aleatória

Relatório de atividade - Processamento Adaptativos de Sinais

Matheus L. T. de Farias

matheuslucas.farias@ee.ufcg.edu.br

Universidade Federal de Campina Grande

Programa de Pós-Graduação em Engenharia Elétrica - PPgEE

Junho de 2026

Campina Grande - PB, Brasil

Resumo—O monitoramento da poluição atmosférica por meio de sensores de material particulado (PM2.5) de baixo custo apresenta desafios relacionados à precisão das medições. Devido a isso, a comunidade acadêmica tem dedicado esforços ao desenvolvimento de técnicas capazes de melhorar a precisão desses dispositivos. Neste trabalho, foi reproduzido um modelo baseado em rede neural *Multilayer Perceptron* (MLP) para a correção temporal desses dados. Além disso, foi aplicado um algoritmo de busca aleatória para a otimização dos hiperparâmetros do modelo proposto na referência. O modelo otimizado alcançou um coeficiente de determinação de $R^2 = 0.944$ no conjunto de teste, superando o resultado reportado pelo estudo de referência ($R^2 = 0.932$), mesmo utilizando uma arquitetura mais simples. Esses resultados demonstram a importância de um processo estruturado de otimização de hiperparâmetros e a eficácia da busca aleatória na obtenção de modelos mais precisos para a calibração de sensores de baixo custo.

Index Terms—Rede neural *Multilayer Perceptron*, Otimização de hiperparâmetros, Busca Aleatória.

I. INTRODUÇÃO

A poluição atmosférica é um grande problema recente devido aos seus impactos negativos sobre a saúde respiratória da população, principalmente aquela causada por partículas finas de material particulado com diâmetro inferior a 2.5 micrômetros (PM2.5), a qual apresenta elevados riscos à vida humana [1].

Dessa forma, esse tipo de poluição recebe atenção especial dos órgãos reguladores, que estabelecem limites máximos para a concentração dessas partículas no ar, de modo a reduzir seus impactos sobre a saúde pública [2].

Diante dessa exigência, o monitoramento frequente e preciso da concentração de PM2.5 torna-se uma prioridade. Entretanto, sistemas de monitoramento de alta precisão apresentam elevado custo financeiro e dependem de estações de medição robustas e complexas. Nesse cenário, surgem os sensores de baixo custo, que podem ser adquiridos em grande quantidade e instalados com maior facilidade, permitindo uma cobertura

espacial e temporal mais ampla. Contudo, esses sensores tendem a apresentar menor precisão e elevada sensibilidade a fatores ambientais, como a umidade do ar [3].

Frente a esse desafio, têm sido propostas abordagens capazes de realizar estimativas precisas da concentração de PM2.5 mesmo utilizando dados provenientes de sensores de baixo custo. Entre essas abordagens, em [4] foi proposta a utilização de uma rede neural do tipo *Multi-Layer Perceptron* (MLP) para realizar a correção temporal desses dados, utilizando medições da cidade de Turim, na Itália, como base de estudo. A abordagem explora a capacidade da rede de aprender relações complexas entre os dados de entrada meteorológicos e o valor de PM2.5.

Entretanto, redes MLP são altamente sensíveis aos hiperparâmetros utilizados durante o treinamento. Como consequência, arquiteturas semelhantes podem produzir resultados significativamente diferentes dependendo da configuração adotada [5].

Dessa forma, uma etapa de ajuste de hiperparâmetros torna-se, muitas vezes, indispensável para a obtenção de um desempenho satisfatório, sendo uma área de pesquisa ativa, com diversos métodos desenvolvidos para esse fim, como a busca aleatória (*Random Search*) [6], que pode utilizar o orçamento computacional de forma mais eficiente do que a busca em grade (*Grid Search*) em problemas nos quais apenas um subconjunto dos hiperparâmetros exerce influência significativa sobre o desempenho do modelo.

A. Objetivos

Diante da problemática apresentada, este trabalho tem como objetivos: (1) reproduzir a metodologia original proposta em [4] para o melhor modelo desenvolvido pelos autores (AirMLP7-1500); e (2) aplicar um algoritmo estruturado de busca aleatória de hiperparâmetros (*Random Search*) com a finalidade de explorar novas configurações e buscar uma

possível melhora no desempenho do modelo originalmente proposto.

II. FUNDAMENTAÇÃO TEÓRICA

Nesta seção são apresentados os conceitos teóricos necessários para o entendimento do trabalho desenvolvido, com foco nas redes neurais do tipo *Multilayer Perceptron* (MLP) e no algoritmo de otimização *Random Search*. Além disso, é apresentada a formulação da principal métrica de avaliação utilizada neste trabalho, o coeficiente de determinação (R^2).

A. Redes Neurais Multilayer Perceptron

As redes MLPs são a estrutura base do desenvolvimento das redes neurais artificiais (RNA). Sua arquitetura consiste no empilhamento de camadas de perceptrons completamente conectadas. Dessa forma, cada perceptron da camada anterior se conecta a todos os perceptrons da camada posterior [5].

Cada perceptron, também conhecido como neurônio, é a unidade fundamental de processamento da rede. Matematicamente, a operação realizada pelo j -ésimo neurônio da camada $i + 1$ é dada pela Equação 1:

$$z_j^{(i+1)} = f \left(\mathbf{w}_j^{(i)T} \mathbf{z}^{(i)} + b_j^{(i+1)} \right) \quad (1)$$

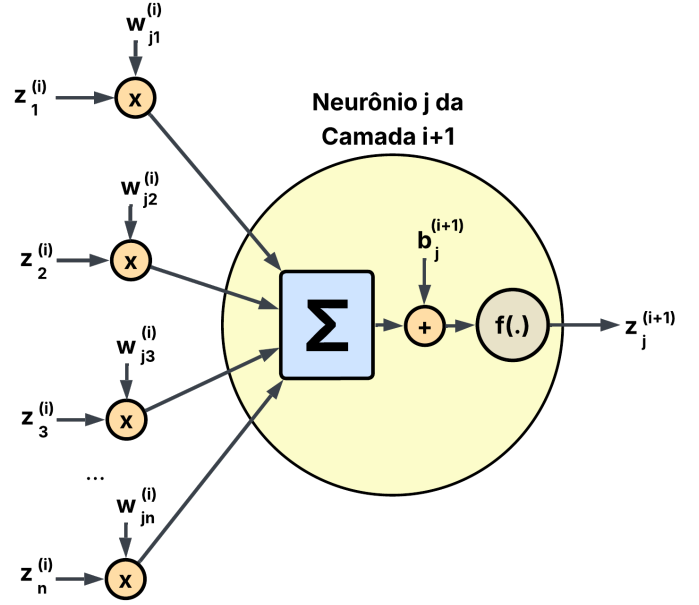
Nessa formulação, $\mathbf{z}^{(i)}$ representa o vetor com as saídas dos neurônios da camada i , que agem como entradas para o neurônio. Esses valores são ponderados individualmente pelo vetor de pesos $\mathbf{w}_j^{(i)}$, que contém as ligações da camada anterior ao j -ésimo neurônio da camada atual. O termo $b_j^{(i+1)}$ consiste no viés (*bias*), que é uma constante adicionada à combinação linear para deslocar o ponto de ativação da rede.

O resultado dessa soma ponderada é passado como argumento para uma função não linear, conhecida como função de ativação $f(\cdot)$, responsável por introduzir não linearidade ao sistema e gerar o valor final da saída do neurônio j da camada $i + 1$, denotado por $z_j^{(i+1)}$ [5]. A estrutura elementar desse arranjo pode ser visualizada na Figura 1.

Em uma rede MLP, há múltiplas unidades de neurônios por camada e múltiplas camadas, característica da qual provém o seu nome. Qualquer arquitetura MLP é estruturada em três tipos de camadas principais: a camada de entrada, à qual são fornecidos os dados externos para processamento; a camada oculta, que pode ser disposta em uma única camada ou em múltiplas camadas empilhadas; e a camada de saída, a qual fornece o resultado final proveniente do processamento de toda a rede [5]. Dessa forma, a topologia típica de uma MLP pode ser ilustrada conforme apresentado na Figura 2.

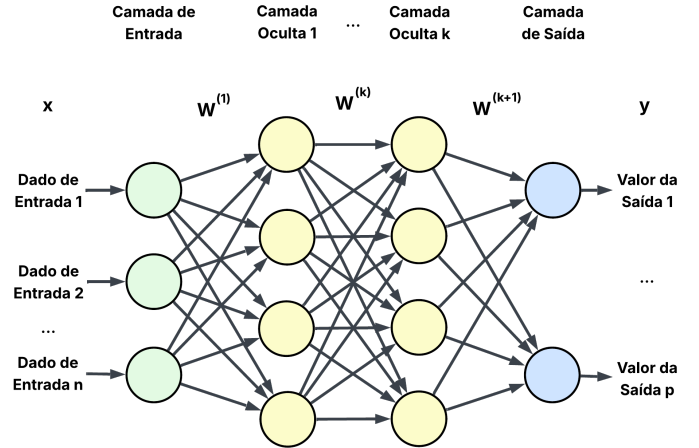
Nesse contexto, $\mathbf{x}$ é o vetor de entradas da rede, $\mathbf{y}$ é o vetor de saída da rede e $\mathbf{W}^{(i)}$ é a matriz de pesos que contempla todas as ligações da camada i para a camada $i + 1$, apresentando a estrutura da Equação 2:

Figura 1: Estrutura elementar de um neurônio



Fonte: Autoria própria

Figura 2: Estrutura geral de uma rede MLP



Fonte: Autoria própria

$$\mathbf{W}^{(i)} = \begin{bmatrix} w_{11}^{(i)} & w_{12}^{(i)} & \cdots & w_{1n_i}^{(i)} \\ w_{21}^{(i)} & w_{22}^{(i)} & \cdots & w_{2n_i}^{(i)} \\ \vdots & \vdots & \ddots & \vdots \\ w_{n_{i+1},1}^{(i)} & w_{n_{i+1},2}^{(i)} & \cdots & w_{n_{i+1},n_i}^{(i)} \end{bmatrix} \in \mathbb{R}^{n_{i+1} \times n_i} \quad (2)$$

na qual cada elemento $w_{jk}^{(i)}$ representa o peso que liga a saída do k -ésimo neurônio da camada i ao j -ésimo neurônio na camada $i + 1$. Dessa forma, o número de linhas da matriz é igual ao número de neurônios na camada $i + 1$ e o número de colunas está relacionado ao número de neurônios na camada i .

A função de ativação é a responsável por fornecer a não linearidade ao sistema, o que permite que a rede aprenda relações não lineares entre a entrada e a saída da rede. As principais funções de ativação utilizadas na literatura são a tangente hiperbólica ($\tanh(\cdot)$), a sigmoide ($\sigma(\cdot)$) e a Unidade Linear Retificada ($ReLU(\cdot)$). Entretanto, existem diversas outras funções de ativação exploradas na literatura.

A função de ativação da camada de entrada é linear e a função de ativação das camadas ocultas costuma ser a mesma para todos os neurônios em todas as camadas. Já a camada de saída costuma ter uma função de ativação diferente dependendo da aplicação, variando se está sendo aplicada a uma tarefa de regressão ou de classificação.

Dessa forma, a saída da camada $i+1$ é dada pela Equação 3:

$$\mathbf{z}^{(i+1)} = f(\mathbf{a}^{(i+1)}) \quad (3)$$

na qual $\mathbf{a}^{(i+1)}$ é a ativação da camada $i+1$, responsável por ativar o neurônio quando utilizada como argumento da função de ativação. Matematicamente, é definida com base na Equação 4:

$$\mathbf{a}^{(i+1)} = \mathbf{W}^{(i)}\mathbf{z}^{(i)} + \mathbf{b}^{(i+1)} \quad (4)$$

com $\mathbf{b}^{(i+1)}$ sendo o vetor de vieses da camada $i+1$.

A passagem dos dados de entrada através das camadas da rede até a obtenção da saída final é denominada propagação direta (*forward propagation*).

O objetivo da rede, no processo de treinamento, é ajustar os pesos e os vieses de tal forma que a saída gerada pela propagação direta seja o mais próxima possível da saída esperada (valor de referência, ou *Ground Truth*). Trata-se de um problema clássico de otimização para determinar os valores dos parâmetros que minimizam uma função de perda (ou de custo). Essa função quantifica a divergência entre a predição da rede e o valor real [5], tal qual expresso na Equação 5:

$$\theta = \arg \min_{\theta} J(\mathbf{x}, \theta, \mathbf{d}) \quad (5)$$

na qual θ representa o vetor contendo todos os parâmetros da rede que se deseja otimizar, $\mathbf{x}$ é a entrada fornecida e $\mathbf{d}$ é a saída de referência desejada.

As principais funções de custo nesse processo são o Erro Quadrático Médio (MSE) e o Erro Absoluto Médio (MAE, ou *LI Loss*), que são amplamente utilizadas em problemas de regressão. Para problemas de classificação, a função de Entropia Cruzada (*Cross-Entropy*) é a mais adotada.

A solução iterativa desse problema é dada a partir do cálculo do gradiente da função de custo em relação aos parâmetros desejados. A atualização ocorre na direção oposta ao gradiente, como explicitado nas Equações 6 e 7:

$$\mathbf{W}^{(i)}[k+1] = \mathbf{W}^{(i)}[k] - \mu \nabla_{\mathbf{W}^{(i)}} J \quad (6)$$

$$\mathbf{b}^{(i)}[k+1] = \mathbf{b}^{(i)}[k] - \mu \nabla_{\mathbf{b}^{(i)}} J \quad (7)$$

nas quais o índice k representa o passo temporal da iteração, μ é a taxa de aprendizado (*learning rate*), um hiperparâmetro entendido como o tamanho do passo que o algoritmo dá na direção negativa da curva de erro, e ∇J é o gradiente da função de custo em relação ao parâmetro em questão.

O cálculo analítico desses gradientes em redes profundas segue a regra da cadeia do cálculo diferencial. Como os gradientes dos parâmetros das camadas mais próximas da saída auxiliam na computação dos gradientes das camadas iniciais, o cálculo ocorre do fim para o início da rede. Essa característica dá origem ao nome do algoritmo: retropropagação do erro, ou *backpropagation* [5].

Na otimização computacional dos parâmetros, vários algoritmos baseados no *backpropagation* estão presentes na literatura, sendo o *Adaptive Moment Estimation* (Adam) um dos mais utilizados devido à sua eficiência empírica.

O processo de treinamento é iterativo. Uma passagem completa de todo o conjunto de dados de treinamento pela rede é denominada época (*epoch*). Durante uma época, os dados de entrada não são propagados todos de uma vez; eles são divididos em subconjuntos (lotes) de tamanho definido *a priori*.

Os hiperparâmetros de uma rede neural são configurações externas ao processo de otimização de pesos e vieses. Diferentemente dos parâmetros internos que são ajustados automaticamente durante o treinamento, os hiperparâmetros servem para delimitar a estrutura arquitetural da rede e definir a dinâmica do algoritmo de aprendizado [5]. Configurações como o número de camadas ocultas, a quantidade de neurônios por camada, a função de ativação, a taxa de aprendizado, o número máximo de épocas e o tamanho dos lotes são exemplos clássicos de hiperparâmetros.

Apesar de não serem atualizados de forma autônoma pelo algoritmo de *backpropagation*, eles são os principais responsáveis pela eficácia e eficiência do modelo. Como as redes neurais costumam ser altamente sensíveis a pequenas variações nesses valores, a busca pela configuração ideal consolidou um campo de pesquisa específico voltado à otimização de hiperparâmetros. Entre as metodologias de busca mais difundidas na literatura, destacam-se a busca em grade (*Grid Search*) e a busca aleatória (*Random Search*).

B. Busca Aleatória (*Random Search*)

Historicamente, a otimização de hiperparâmetros é realizada predominantemente por meio de busca manual ou da busca em grade (*Grid Search*). A busca manual baseia-se na intuição e na experiência do projetista, consistindo em um processo empírico de tentativa e erro que se torna exaustivo e pouco sistemático em arquiteturas complexas. Já a busca em grade é uma abordagem estruturada que avalia todas as combinações possíveis dentro de um conjunto discreto e predefinido de valores. Embora rigorosa, essa técnica sofre severamente com a dimensionalidade, pois o tempo de processamento cresce exponencialmente à medida que novos hiperparâmetros são adicionados.

A busca aleatória surge como uma alternativa para contornar essas limitações. Trata-se de um método estocástico de otimização no qual combinações de hiperparâmetros são amostradas independentemente a partir de distribuições de probabilidade ou intervalos definidos pelo projetista. Essa estratégia é particularmente eficaz em problemas de otimização de hiperparâmetros, pois, conforme demonstrado por Bergstra e Bengio [6], apenas um subconjunto dos hiperparâmetros costuma exercer influência significativa sobre o desempenho de modelos de aprendizado de máquina, caracterizando um problema de baixa dimensionalidade efetiva.

Além disso, o *Random Search* apresenta vantagens práticas relevantes. O método é inerentemente paralelizável, uma vez que cada amostragem e treinamento subsequente constituem processos independentes, permitindo sua execução simultânea em múltiplos núcleos de processamento. Adicionalmente, a técnica oferece controle direto sobre o orçamento computacional, podendo ser interrompida a qualquer momento ou configurada para executar um número predeterminado de iterações. Dessa forma, garante-se a obtenção da melhor configuração encontrada dentro das restrições de tempo e recursos disponíveis.

Do ponto de vista operacional, o algoritmo funciona por meio de sucessivas amostragens do espaço hiperparamétrico. Em cada iteração, seleciona-se aleatoriamente um conjunto de hiperparâmetros, sendo amostrado um valor para cada parâmetro dentro dos limites previamente definidos. A partir dessa configuração, constrói-se a arquitetura da rede e definem-se os padrões de treinamento, executando-se então o processo de aprendizado. Após o treinamento, o desempenho do modelo é avaliado utilizando um conjunto de dados distinto daquele empregado durante o ajuste dos parâmetros. Esse conjunto, denominado conjunto de validação, permite estimar a capacidade de generalização do modelo para as configurações de hiperparâmetros selecionadas.

A partir das previsões produzidas pela rede, calcula-se o valor da função de custo correspondente, o qual é armazenado para posterior comparação. Em seguida, reinicia-se o processo até que seja atingido o número máximo de execuções ou outro critério de parada previamente estabelecido. Ao término, avalia-se qual conjunto de hiperparâmetros produziu o melhor resultado para a função de custo adotada, sendo este selecionado como a melhor configuração encontrada dentre as alternativas analisadas.

Uma desvantagem desse algoritmo é que nem todas as combinações possíveis são exploradas, justamente em razão de sua natureza aleatória. Como consequência, o método não garante a identificação do ótimo global do espaço hiperparamétrico, mas apenas do melhor conjunto dentre aqueles efetivamente avaliados. Ainda assim, frequentemente apresenta soluções competitivas com um custo computacional significativamente menor do que abordagens exaustivas, como a busca em grade.

C. Coeficiente de Determinação (R^2)

O coeficiente de determinação, representado por R^2 , é uma métrica amplamente utilizada para avaliar o desempenho de modelos de regressão. Essa métrica quantifica a proporção da variabilidade dos dados observados que pode ser explicada pelo modelo preditivo [7].

O valor de R^2 é calculado por meio da Equação 8.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} \quad (8)$$

em que y_i representa o valor observado, $\hat{y}_i$ corresponde ao valor estimado pelo modelo e $\bar{y}$ representa a média dos valores observados.

O coeficiente de determinação assume valor máximo igual a 1, indicando que o modelo é capaz de explicar completamente a variabilidade dos dados. Valores próximos de 1 indicam elevado poder preditivo, enquanto valores próximos de 0 sugerem desempenho semelhante ao obtido pela utilização da média dos dados observados. Valores negativos podem ocorrer quando o modelo apresenta desempenho inferior ao de uma predição baseada apenas na média [7].

III. BASE DE DADOS

Para modelos baseados em dados, como é o caso das MLPs, a base de dados utilizada para o treinamento e a validação dos modelos é um objeto de extremo interesse na implementação dos algoritmos, sendo parte fundamental para a eficácia dos modelos treinados.

Para o treinamento e validação dos modelos desenvolvidos em [4], os autores utilizaram os dados disponibilizados no repositório [8].

A base de dados utilizada conta com medições de 5 dispositivos de baixo custo do modelo Arianna, produzidos pela Wiseair. Cada dispositivo é equipado com um sensor de temperatura, um sensor de umidade relativa do ar e um sensor SPS30, responsável por medir concentrações de PM1, PM2.5, PM4 e PM10. Além dos dados dos dispositivos de baixo custo, também há dados de uma estação de referência gerenciada pela Arpa Piemonte, que fornece medições de material particulado mais precisas e atua no trabalho como *Ground Truth* para os valores de PM2.5 esperados.

Os dispositivos Arianna foram posicionados em áreas verdes, distantes das vias mais congestionadas, na cidade de Turim, na Itália. Os dados disponíveis cobrem quatro meses e estão divididos em dois períodos de operação: o primeiro ocorreu entre os meses de março e abril, no qual 3 dispositivos ficaram em operação; o segundo ocorreu entre outubro e novembro, com os outros 2 dispositivos em funcionamento. A estação de referência esteve operante durante ambos os períodos.

Em relação à taxa de amostragem, os dispositivos de baixo custo realizam quatro medições por hora (uma a cada 15 minutos), enquanto a estação de referência fornece apenas um valor de medição a cada 1 hora de operação.

Dentre as grandezas disponíveis, foram utilizadas as seguintes para o treinamento e a validação dos modelos:

- Concentração de PM2.5 [$\mu\text{g}/\text{m}^3$]
- Umidade relativa do ar [%]
- Temperatura [$^{\circ}\text{C}$]
- Pressão atmosférica [hPa]
- Cobertura de nuvens [%]
- Velocidade do vento [m/s]

As três primeiras grandezas são obtidas pelo próprio dispositivo Arianna, enquanto as outras três são provenientes de dados meteorológicos adicionais.

Essas grandezas são utilizadas pelos modelos como variáveis de entrada, enquanto o valor de PM2.5 medido pela estação de referência é utilizado como o valor de saída esperado da rede.

IV. IMPLEMENTAÇÃO ORIGINAL

Nesta seção é apresentada a implementação original proposta em [4] e reproduzida neste trabalho. Com isso, evidenciam-se as etapas de pré-processamento dos dados, a busca empírica de hiperparâmetros realizada pelos autores, a arquitetura do melhor modelo encontrado e como se deu a reprodução no escopo deste trabalho.

A. Pré-processamento

O pré-processamento de dados realizado pelos autores em [4] consistiu, inicialmente, na remoção de dados ausentes da base de dados, descartando-se toda linha em que ao menos uma das grandezas dos sensores não estivesse disponível.

Na sequência, abordou-se a disparidade nas taxas de amostragem. Uma vez que os sensores de baixo custo registram uma amostra a cada 15 minutos e a estação de referência fornece apenas uma medição por hora, tornou-se incompatível relacionar diretamente as entradas da rede com os valores esperados. Para solucionar esse problema, os autores optaram por uma estratégia de aumento de dados (*data augmentation*), replicando o valor de referência. Dessa forma, para as quatro medições realizadas pelos sensores dentro de uma determinada hora, foi atribuído o mesmo valor de referência fornecido pela estação naquele intervalo, conforme ilustrado na Figura 3.

O último tratamento realizado foi o janelamento das entradas. Os autores constataram que a predição do valor atual de PM2.5 possui maior relação com um conjunto de amostras históricas recentes do que apenas com a medição instantânea. Como as redes MLP não lidam nativamente com dependências temporais, a solução adotada foi fornecer à camada de entrada, de forma achatada, um conjunto de amostras passadas consecutivas de cada grandeza. O valor de saída esperado pela rede foi definido como a medição de PM2.5 correspondente ao último instante dessa janela de tempo. A Figura 4 evidencia como essa estruturação temporal foi implementada.

Após a conclusão do pré-processamento, o conjunto de dados foi dividido em subconjuntos de treinamento e teste por

meio de amostragem aleatória, utilizando-se uma proporção de 75% para treinamento e 25% para teste.

B. Busca de Hiperparâmetros

Em [4], os autores também dedicaram esforços à otimização de hiperparâmetros. Entretanto, a busca realizada consistiu em uma abordagem híbrida entre o método empírico manual e a busca em grade, o que reduziu a sistematização rigorosa do experimento. Além disso, a metodologia original omitiu o uso de um conjunto de dados de validação independente durante o processo de sintonia, utilizando diretamente o conjunto de teste para selecionar a melhor arquitetura. Essa prática pode ter enviesado a escolha dos parâmetros e superestimado os resultados na avaliação final.

A Tabela I resume os hiperparâmetros que fizeram parte dessa etapa no estudo original, bem como o espaço de busca correspondente estipulado pelos autores. A busca ocorreu em duas fases: uma varredura inicial combinando tamanho de lote, Tamanho de janela e neurônios (até 700), seguida por uma exploração empírica posterior testando redes mais largas (até 1500 neurônios) fixando os melhores parâmetros da etapa anterior.

Tabela I: Espaço de busca de hiperparâmetros avaliado no modelo original

Hiperparâmetro	Valores Explorados
Número de camadas	6, 7 e 8
Neurônios por camada	300, 500, 700, 900, 1100 e 1500
Tamanho da Janela	6, 12 e 20 amostras
Tamanho do lote	64, 256 e 512

Fonte: Adaptado de [4]

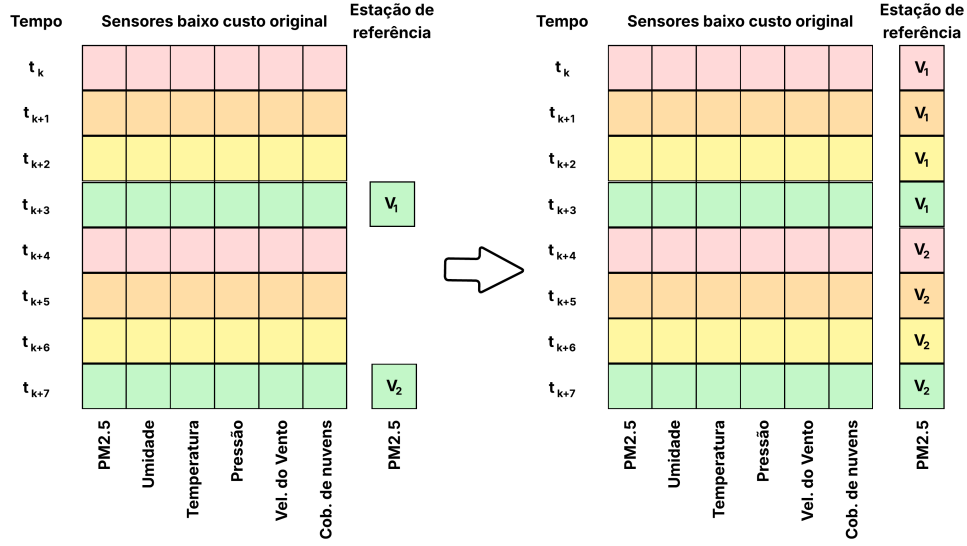
C. Reprodução do melhor modelo original

Ao finalizar o processo de busca de hiperparâmetros, os autores determinaram como modelo ideal aquele que apresentou o melhor desempenho segundo a métrica R^2 . O modelo de maior destaque foi intitulado AirMLP7-1500, o qual obteve como resultado final $R^2 = 0.932$ no conjunto de teste.

A Tabela II apresenta a configuração exata dos hiperparâmetros encontrados no processo de busca que definem essa arquitetura, bem como os demais parâmetros de treinamento adotados pelos autores e reproduzidos no escopo deste trabalho.

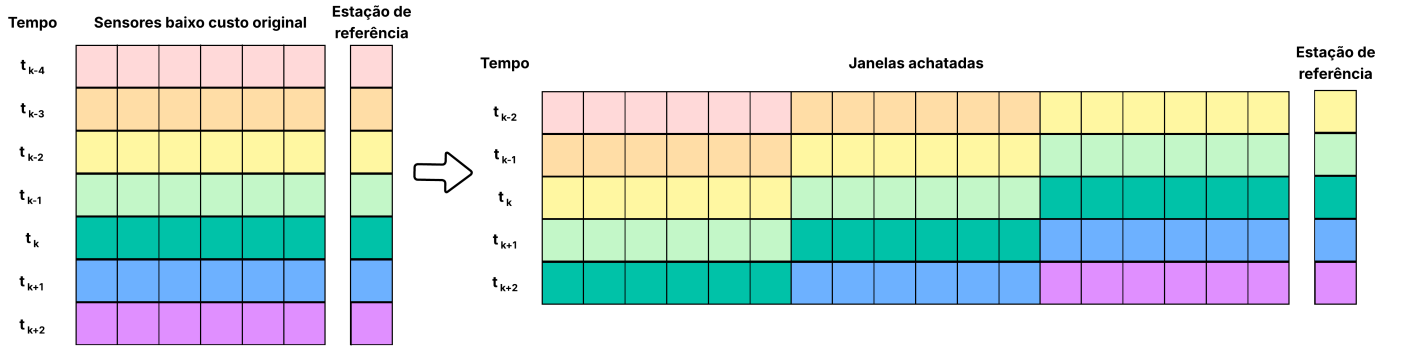
Os autores não realizaram uma normalização prévia dos dados de entrada. Em vez disso, incluíram uma camada de normalização de lote (*Batch Normalization*) logo após a camada de entrada da rede. Essa camada normaliza, para cada lote, as ativações associadas a cada característica, de modo que apresentem média aproximadamente nula e variância unitária. Esse procedimento contribui para estabilizar o treinamento, permitindo o uso de taxas de aprendizado mais elevadas e reduzindo a sensibilidade da otimização a variações nas distribuições das ativações ao longo do tempo.

Figura 3: Estratégia de aumento de dados (*data augmentation*) para equalização da amostragem



Fonte: Adaptado de [4]

Figura 4: Janelamento temporal dos dados de entrada para uma janela exemplo de 3 amostras



Fonte: Adaptado de [4]

Tabela II: Configuração e hiperparâmetros do melhor modelo original (AirMLP7-1500)

Hiperparâmetro	Valor adotado
Número de camadas	7
Neurônios por camada	1500
Janelamento temporal	20 amostras
Tamanho do lote	64
Função de ativação	ReLU
Função de perda	L1 (Erro Absoluto Médio)
Número de épocas	200
Otimizador	RAadam
Taxa de aprendizado	$5 \cdot 10^{-5}$

Fonte: Adaptado de [4]

Diferente das demais, a camada de saída possui como função de ativação a função linear.

Com os hiperparâmetros definidos e os métodos consolida-

dos, realizou-se a reprodução do modelo seguindo os passos descritos nesta seção. Para a implementação computacional, utilizou-se a linguagem Python em conjunto com as bibliotecas TensorFlow e Keras, abrangendo tanto a construção arquitetural quanto o treinamento do modelo.

O pré-processamento dos dados e os hiperparâmetros estruturais foram implementados estritamente conforme especificado no artigo original. A única exceção concentrou-se no algoritmo otimizador empregado: em vez de utilizar o RAadam, conforme proposto pelos autores no código-fonte, optou-se pelo uso do otimizador Adam convencional. Essa decisão foi motivada pela ausência de uma implementação nativa e estável do RAadam nas versões padrão das bibliotecas utilizadas. Considerando que ambos os algoritmos pertencem à mesma família de métodos de otimização adaptativa e que as diferenças de convergência tendem a ser marginais em diversos cenários práticos, a utilização do Adam foi considerada uma aproximação robusta e adequada para a reprodução da

metodologia.

V. OTIMIZAÇÃO UTILIZANDO BUSCA ALEATÓRIA

Os autores em [4] aplicaram um método de otimização de hiperparâmetros, como abordado na seção anterior. Entretanto, o método empregado careceu de robustez e estruturação, apresentando uma falha metodológica grave: a ausência de um conjunto de dados de validação durante o processo de sintonia. Consequentemente, o modelo apontado como o ideal pelos autores pode consistir em uma versão enviesada e não corresponder ao melhor modelo possível dentro do espaço hiperparamétrico delimitado.

Dessa forma, o presente trabalho dedicou esforços à implementação de um método de otimização de hiperparâmetros dotado de maior robustez e com uma estruturação mais adequada para esse tipo de problema. Esta seção aborda a metodologia adotada na implementação dessa etapa de otimização.

Para viabilizar a otimização de forma correta, foi necessário reorganizar a separação original da base de dados, dividindo-a em três subconjuntos: treinamento, validação e teste. A fim de não reduzir o volume de dados de aprendizado em relação ao artigo original, manteve-se a proporção de 75% dos dados totais para treinamento, enquanto os 25% restantes foram divididos igualmente, resultando em 12,5% para validação e 12,5% para teste. A separação dos dados foi realizada de forma aleatória, seguindo a mesma abordagem proposta na referência.

O espaço hiperparamétrico adotado nesta implementação é composto por 6 hiperparâmetros distintos, cada um com um intervalo de busca próprio e relativamente extenso. Considerando que métodos tradicionais, como a busca em grade, sofrem severamente com o aumento do número de combinações de hiperparâmetros nesse cenário, optou-se pela utilização da busca aleatória. Conforme discutido por [6], esse método pode explorar o espaço de hiperparâmetros de forma mais eficiente do que a busca em grade, especialmente quando apenas um subconjunto dos hiperparâmetros exerce influência significativa sobre o desempenho do modelo.

O processo de otimização foi estruturado em duas etapas: na primeira, explorou-se um espaço hiperparamétrico amplo; na segunda, realizou-se um estreitamento desse espaço a fim de refinar a busca com base nas melhores regiões identificadas na etapa anterior. Ambas as etapas foram executadas com um limite de 400 iterações cada, garantindo uma exploração abrangente do espaço de hiperparâmetros.

Para otimizar o tempo computacional, adotou-se um critério de parada antecipada (*Early Stopping*) durante o treinamento das redes avaliadas. Em cada iteração da busca, o modelo foi treinado com uma paciência de 15 épocas. Isso significa que, se a função de perda avaliada no conjunto de validação não apresentasse melhoria ao longo de 15 épocas consecutivas, o treinamento era interrompido. Essa estratégia evitou que redes com configurações ruins completassem as 200 épocas de treinamento sem melhora no erro.

Outra alteração estrutural importante em relação à metodologia original foi a eliminação da camada de normalização de lote. Em substituição, aplicou-se um processo de normalização prévia aos dados, assegurando que as variáveis de entrada e saída apresentassem média nula e variância unitária (*Standard Scaler*). Essa modificação fez-se necessária para garantir compatibilidade e estabilidade numérica no treinamento, uma vez que o novo espaço de busca testa funções de ativação distintas, cujo desempenho poderia ser prejudicado pela escala dos dados.

As demais características arquiteturais não comentadas nesta seção foram mantidas estritamente conforme adotadas na referência [4]. A Tabela III detalha os hiperparâmetros e as faixas de valores explorados nas Etapas 1 e 2 do processo de busca aleatória.

Os intervalos de busca da primeira etapa foram definidos com base nos hiperparâmetros explorados em [4], buscando simultaneamente preservar a coerência com o estudo original e ampliar a capacidade exploratória da otimização. O número de camadas ocultas foi expandido para valores inferiores e superiores aos avaliados pelos autores, permitindo investigar arquiteturas mais rasas e mais profundas. Para a quantidade de neurônios por camada, optou-se por valores convencionalmente empregados em aplicações de redes MLP, reduzindo a complexidade em relação ao modelo AirMLP7-1500 e possibilitando a avaliação de arquiteturas mais compactas. Os tamanhos de janela foram escolhidos em torno do valor que apresentou melhor desempenho na referência, enquanto os tamanhos de lote seguiram o mesmo espaço de busca adotado pelos autores. Em relação às funções de ativação, foram consideradas três alternativas clássicas da literatura: *ReLU*, tangente hiperbólica (*tanh*) e sigmoide (σ). Por fim, a taxa de aprendizado foi amostrada segundo uma distribuição log-uniforme, abrangendo desde o valor utilizado na referência até valores significativamente maiores, permitindo uma exploração ampla desse hiperparâmetro.

O desempenho das configurações avaliadas na Etapa 1 foi mensurado por meio do coeficiente de determinação (R^2), adotando o mesmo critério utilizado em [4]. A análise das regiões associadas aos maiores valores de R^2 permitiu identificar tendências favoráveis no espaço hiperparamétrico. Com base nesses resultados, definiu-se um espaço de busca mais restrito para a Etapa 2, concentrando a exploração nas regiões que demonstraram maior potencial de desempenho.

VI. RESULTADOS

Com a reprodução do artigo original, a execução da busca aleatória e a avaliação do melhor modelo determinado pelo método proposto, foram obtidos os resultados deste trabalho. Esses resultados serviram para analisar a eficácia e a eficiência das implementações, bem como para comparar os diferentes modelos avaliados.

Tabela III: Espaço de busca de hiperparâmetros adotado para a otimização via Busca Aleatória

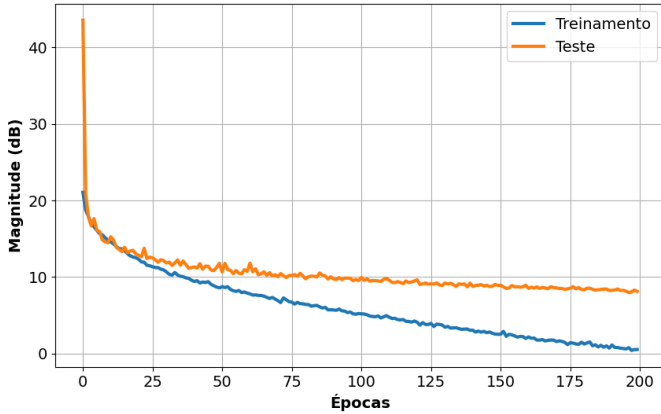
Hiperparâmetro	Valores - Etapa 1 (Exploração)	Valores - Etapa 2 (Refinamento)
Número de camadas ocultas	4 a 10	4 a 7
Neurônios por camada	64, 128, 256 e 512	256, 512 e 1024
Janelamento temporal	10, 20 e 30	20 e 30
Tamanho do lote	64, 128, 256 e 512	64 e 128
Função de ativação	ReLU, tanh e σ	ReLU e tanh
Taxa de aprendizado	LogUniform(10^{-5} , 10^{-2})	LogUniform(10^{-4} , $10^{-2.5}$)

Fonte: Autoria própria

A. Resultado da reprodução

O primeiro resultado foi obtido ao final do treinamento da rede neural e consiste na análise da evolução do erro ao longo das épocas de treinamento. A Figura 5 apresenta as curvas de erro por época para os conjuntos de treinamento e teste.

Figura 5: Evolução do erro de treinamento e teste durante o treinamento



Fonte: Autoria Própria

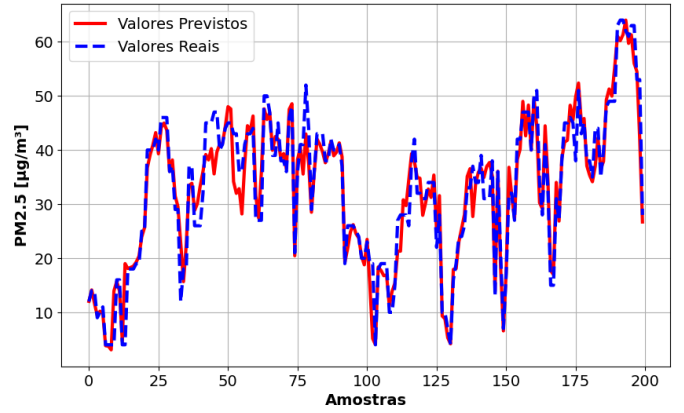
Por meio desse gráfico, observa-se que o erro de treinamento decresce mais rapidamente do que o erro de teste, comportamento esperado, uma vez que o modelo aprende a partir dos dados pertencentes ao conjunto de treinamento. Entretanto, nota-se que a curva correspondente ao erro de teste apresenta uma redução mais lenta e quase constante ao longo das épocas. Esse comportamento pode indicar uma dificuldade do modelo em generalizar para dados não observados e, possivelmente, o início de um processo de sobreajuste.

Após o treinamento, a rede foi utilizada para estimar a concentração de PM2.5 utilizando os dados do conjunto de teste. A Figura 6 apresenta a comparação entre os valores reais e estimados para as 200 primeiras amostras.

Qualitativamente, verifica-se uma aderência consistente entre a curva estimada e a curva real, indicando que o modelo foi capaz de prever de forma satisfatória o comportamento da concentração de PM2.5 ao longo do tempo.

Para uma avaliação quantitativa, foi calculada a métrica de coeficiente de determinação (R^2) considerando toda a série de previsões. O valor obtido foi de 0.92. Embora esse resultado

Figura 6: Estimativa de PM2.5 para as 200 primeiras amostras do conjunto de teste.



Fonte: Autoria Própria

seja inferior ao reportado no artigo original para o mesmo modelo, deve-se considerar que a implementação realizada não foi idêntica à descrita pelos autores, principalmente em razão da utilização de um otimizador diferente e do ambiente de desenvolvimento adotado (TensorFlow/Keras). Dessa forma, o valor alcançado pode ser considerado suficientemente próximo ao apresentado no trabalho original, indicando que a reprodução do modelo foi realizada de maneira satisfatória.

B. Resultado da Otimização

Com a execução da primeira etapa de otimização, foram analisadas 400 combinações de hiperparâmetros, das quais foram obtidos diversos resultados relevantes.

Para fins de comparação quantitativa, a Tabela IV resume os 10 melhores conjuntos de hiperparâmetros encontrados, utilizando como métrica principal o coeficiente de determinação (R^2) obtido nos dados de validação.

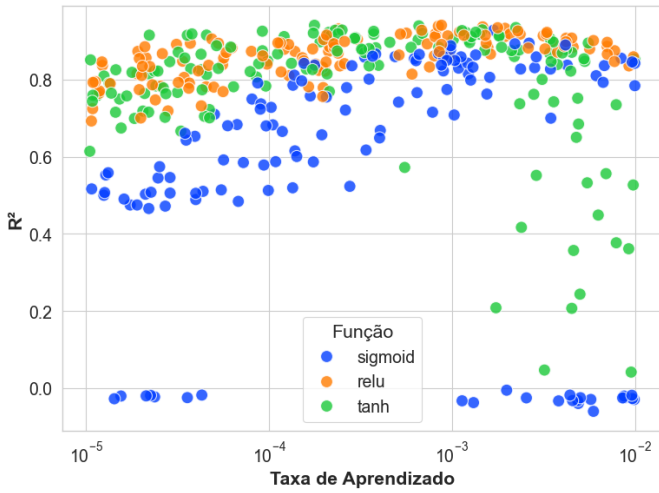
Além disso, foram elaborados alguns gráficos que auxiliaram na análise qualitativa do processo de otimização. A Figura 7 mostra como a taxa de aprendizado e a função de ativação afetam a métrica de R^2 .

Já a Figura 8 apresenta um mapa de calor relacionando o número de neurônios por camada e o número de camadas

Tabela IV: Melhores configurações obtidas na busca aleatória na Etapa 1.

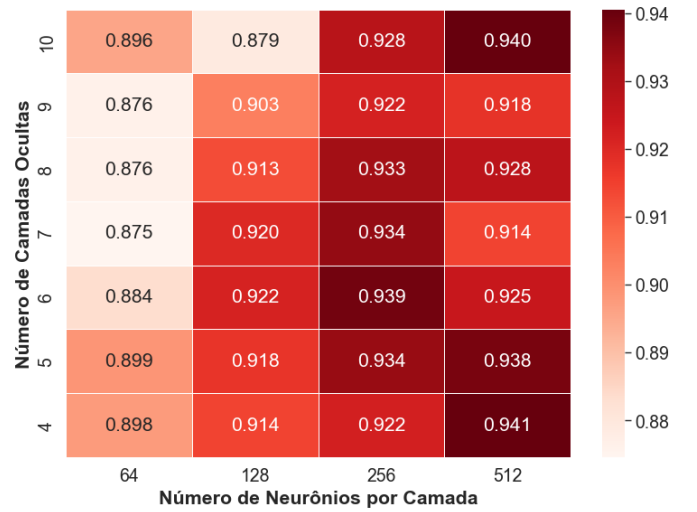
Janelas	Camadas Ocultas	Neurônios	Função	T. aprendizado	Lote	R^2
30	4	512	ReLU	0.000875	64	0.940581
20	10	512	tanh	0.000177	64	0.940359
20	6	256	tanh	0.000659	64	0.938860
20	5	512	ReLU	0.000841	128	0.938172
20	4	512	ReLU	0.001470	512	0.936278
20	7	256	tanh	0.002228	256	0.934454
20	5	256	ReLU	0.001776	256	0.934043
20	8	256	ReLU	0.001947	512	0.932519
30	4	512	ReLU	0.000239	64	0.930825
20	5	256	tanh	0.001461	64	0.928518

Figura 7: Influência da taxa de aprendizado e da função de ativação na métrica R^2 na Etapa 1



Fonte: Autoria Própria

Figura 8: Valor máximo de R^2 por combinação entre número de neurônios e número de camadas ocultas na Etapa 1



Fonte: Autoria Própria

ocultas, avaliando o valor máximo de R^2 obtido para cada combinação.

Por fim, a Figura 9 apresenta dois gráficos do tipo *boxplot*, que avaliam como o tamanho da janela e o tamanho do lote influenciam o valor de R^2 .

Com base nesses resultados, observa-se que a profundidade da rede não exerce influência significativa sobre o desempenho do modelo, uma vez que configurações com menor número de camadas apresentaram resultados competitivos e, em alguns casos, superiores aos obtidos por redes mais profundas. Dessa forma, para a segunda etapa, optou-se por reduzir o espaço de busca, diminuindo o número de camadas ocultas analisadas.

Além disso, o número de neurônios mostrou-se um fator fortemente relacionado ao aumento do desempenho da rede, indicando que arquiteturas com maior capacidade representacional tendem a produzir melhores resultados. Dessa forma, os melhores modelos apresentaram entre 256 e 512 neurônios por camada. Assim, na segunda etapa, o espaço de busca foi ajustado para incluir configurações com um número ainda maior de neurônios.

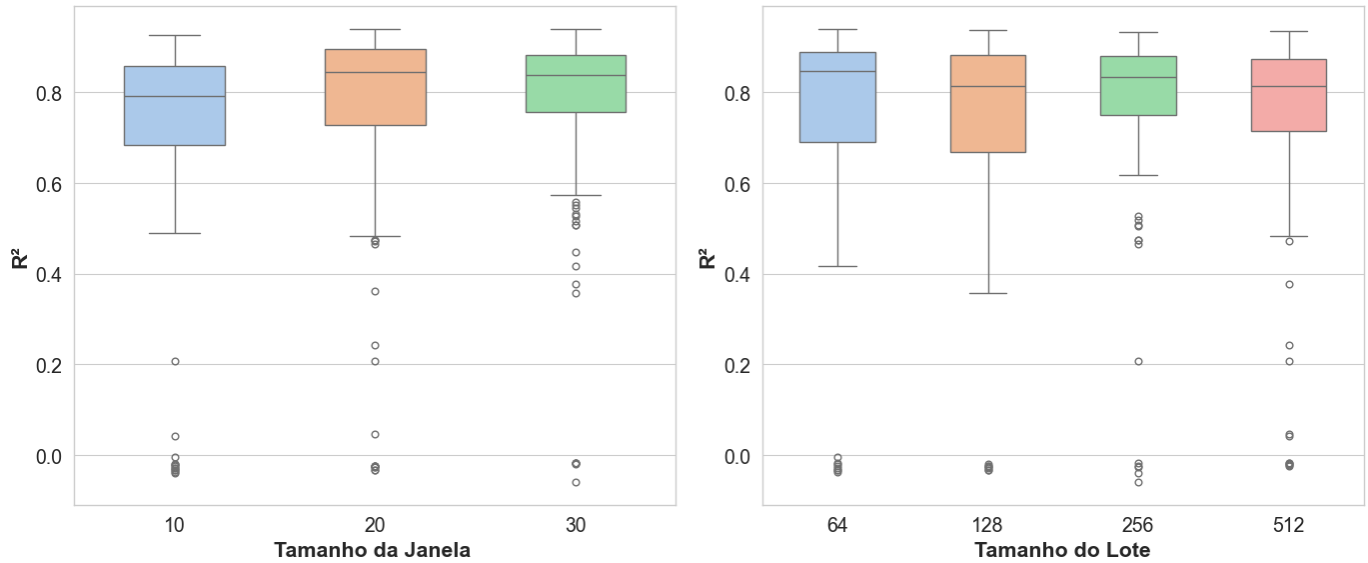
Em relação à taxa de aprendizado, observa-se que valores excessivamente baixos ou elevados comprometem o desempenho do modelo. A região de melhor desempenho concentra-se aproximadamente entre 1×10^{-4} e 1×10^{-3} , justificando a redução do espaço de busca para essa faixa de valores.

Quanto às funções de ativação, verificou-se que os modelos de pior desempenho utilizaram predominantemente a função sigmoid, enquanto os melhores resultados foram obtidos com as funções ReLU e tanh. Entre essas duas alternativas, a ReLU apresentou menor variabilidade e desempenho ligeiramente superior, o que motivou a exclusão da função sigmoid do espaço de busca da segunda etapa.

Por fim, observou-se que modelos com janelas maiores apresentaram melhor desempenho, o que justificou a remoção da janela de tamanho 10 do espaço de busca. Da mesma forma, modelos treinados com tamanhos de lote menores obtiveram resultados superiores, levando à concentração da busca em valores menores para esse hiperparâmetro na etapa seguinte.

Com a análise da Etapa 1 finalizada, passou-se para a

Figura 9: *Boxplots* da relação entre o tamanho da janela e o tamanho do lote com o valor de R^2 na Etapa 1

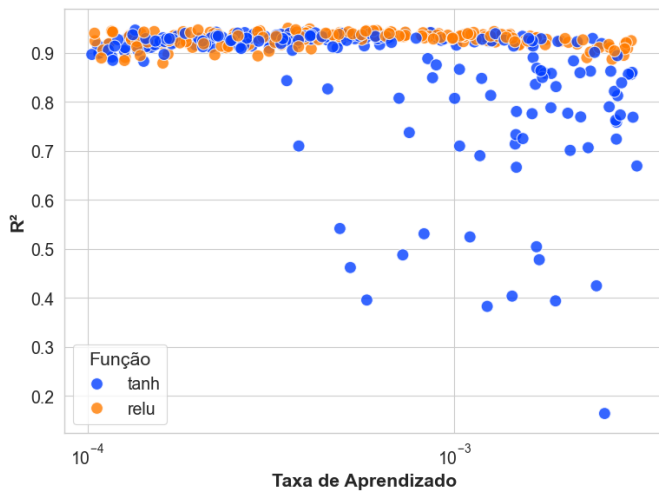


Fonte: Autoria Própria

segunda etapa da busca aleatória, na qual os espaços de busca dos hiperparâmetros foram refinados. A Tabela V apresenta os 10 melhores conjuntos de hiperparâmetros obtidos nessa etapa.

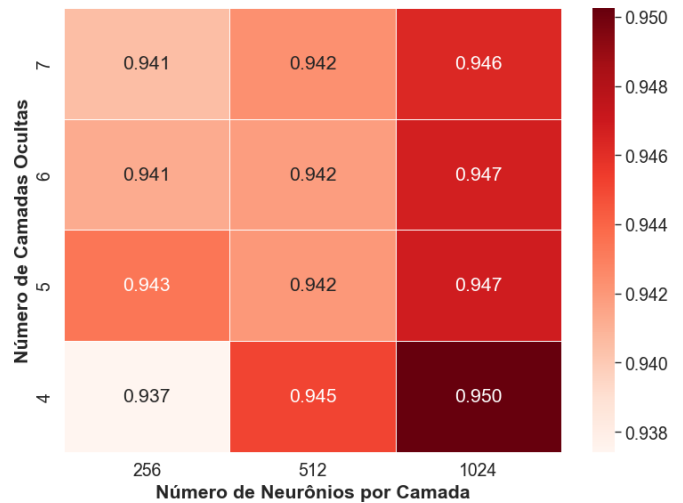
Os gráficos auxiliares referentes à taxa de aprendizado e à função de ativação são apresentados na Figura 10. O mapa de calor contendo os melhores valores de R^2 para cada combinação entre número de camadas ocultas e número de neurônios por camada encontra-se na Figura 11. Já os *boxplots* que relacionam o tamanho da janela e o tamanho do lote com o desempenho dos modelos são apresentados na Figura 12.

Figura 10: Influência da taxa de aprendizado e da função de ativação na métrica R^2 na Etapa 2



Fonte: Autoria Própria

Figura 11: Valor máximo de R^2 por combinação entre número de neurônios e número de camadas ocultas na Etapa 2



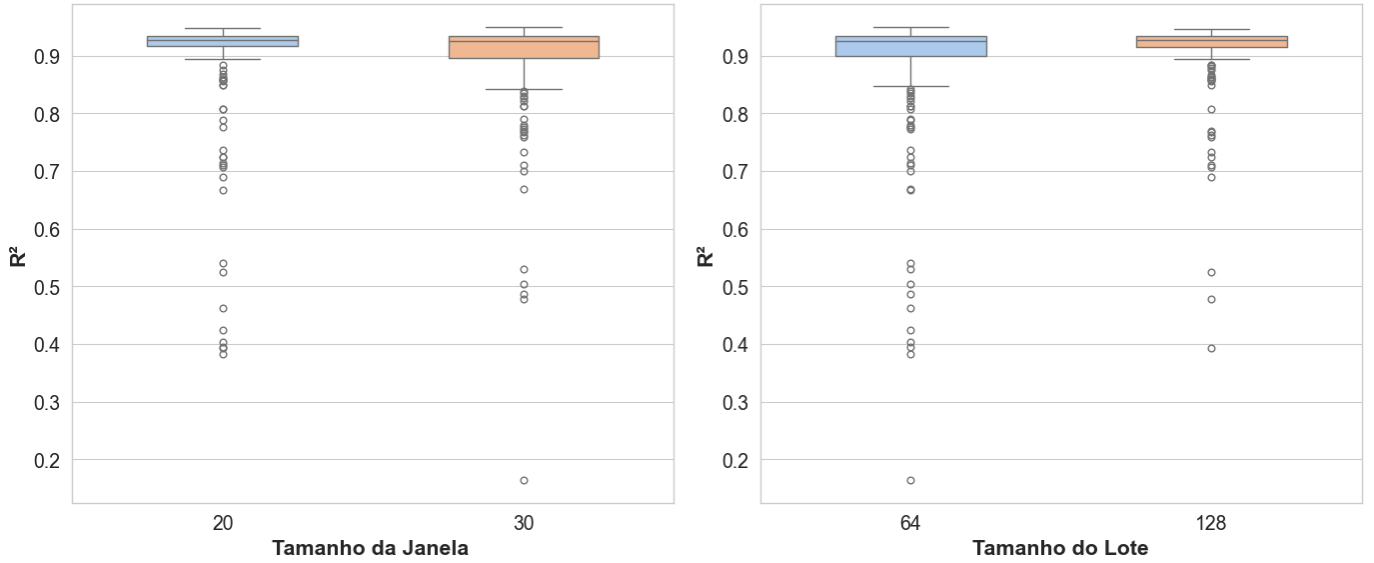
Fonte: Autoria Própria

A partir desses resultados, é possível realizar uma análise mais detalhada dos hiperparâmetros avaliados. Inicialmente, observa-se que taxas de aprendizado mais elevadas tendem a provocar maior instabilidade no treinamento, resultando em maior variabilidade dos valores de R^2 . Em contrapartida, a função de ativação ReLU demonstrou maior robustez às variações da taxa de aprendizado, destacando-se como a alternativa mais adequada para a aplicação.

Outra conclusão relevante refere-se à profundidade da rede neural. Os resultados indicam que o aumento do número de

Tabela V: Melhores configurações obtidas na busca aleatória na Etapa 2

Janelas	Camadas Ocultas	Neurônios	Função	T. aprendizado	Lote	R^2
30	4	1024	ReLU	0.000352	64	0.950253
20	4	1024	ReLU	0.000403	64	0.948627
30	5	1024	ReLU	0.000369	128	0.946890
30	6	1024	ReLU	0.000200	128	0.946729
30	7	1024	tanh	0.000135	64	0.946323
20	5	1024	ReLU	0.000614	128	0.945404
20	4	512	ReLU	0.000577	64	0.945091
20	5	1024	ReLU	0.000601	128	0.945046
30	4	1024	ReLU	0.000295	64	0.944894
20	4	1024	ReLU	0.000573	128	0.944315

Figura 12: *Boxplots* da relação entre o tamanho da janela e o tamanho do lote com o valor de R^2 na Etapa 2

Fonte: Autoria Própria

camadas ocultas não produz ganhos expressivos de desempenho, uma vez que arquiteturas mais rasas e mais profundas apresentaram valores de R^2 bastante próximos. Por outro lado, a quantidade de neurônios por camada mostrou-se um fator mais influente, sendo observado que redes com maior número de neurônios tendem a alcançar melhores resultados.

Em relação ao tamanho da janela de entrada e ao tamanho do lote, os resultados mostraram-se pouco discriminativos. Para as janelas de 20 e 30 amostras, observa-se desempenho semelhante, com distribuições de R^2 bastante próximas, sem evidências de ganhos consistentes decorrentes do aumento da janela. De forma análoga, os tamanhos de lote de 64 e 128 amostras apresentaram comportamento semelhante, não indicando vantagem significativa de uma configuração sobre a outra. Dessa forma, dentro dos intervalos avaliados, o desempenho do modelo mostrou-se relativamente insensível às variações desses hiperparâmetros.

C. Resultado da Implementação do Melhor Modelo

Após a etapa de otimização, o conjunto de melhores hiperparâmetros foi adotado para a implementação do melhor modelo encontrado durante a busca aleatória. A Tabela VI apresenta os hiperparâmetros utilizados, incluindo tanto aqueles definidos durante o processo de otimização quanto aqueles mantidos conforme a configuração proposta na referência utilizada.

Assim como durante o processo de otimização, os dados foram divididos em conjuntos de treinamento, validação e teste, utilizando a proporção de 75%, 12,5% e 12,5%, respectivamente. Além disso, foi aplicada uma normalização dos dados de entrada para média nula e variância unitária. Por fim, o treinamento foi conduzido utilizando o mecanismo de parada antecipada com paciência de 15 épocas, a fim de reproduzir as mesmas condições empregadas durante a busca aleatória.

Com essa configuração, o treinamento ocorreu durante 185 épocas, sendo que o melhor desempenho sobre os dados de validação foi obtido na época 170. A Figura 13 apresenta a

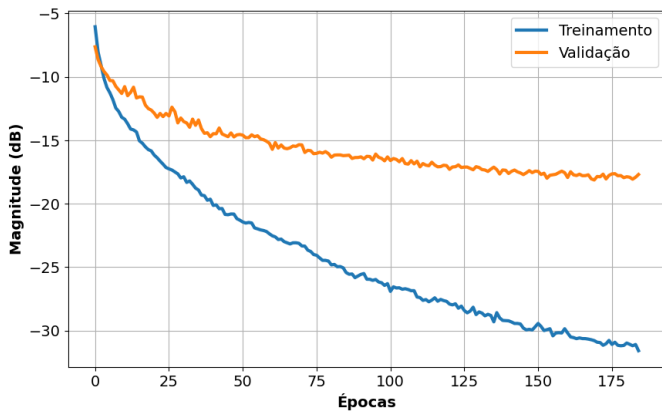
Tabela VI: Configuração e hiperparâmetros do melhor modelo definido pela busca aleatória

Hiperparâmetro	Valor adotado
Número de camadas ocultas	4
Neurônios por camada	1024
Janelamento temporal	30 amostras
Tamanho do lote	64
Função de ativação	ReLU
Função de perda	Erro Absoluto Médio
Número de épocas	200
Otimizador	Adam
Taxa de aprendizado	3.52×10^{-4}

Fonte: Autoria Própria

evolução da função de perda para os conjuntos de treinamento e validação ao longo das épocas.

Figura 13: Evolução do erro de treinamento e validação durante o treinamento do melhor modelo encontrado na busca aleatória



Fonte: Autoria Própria

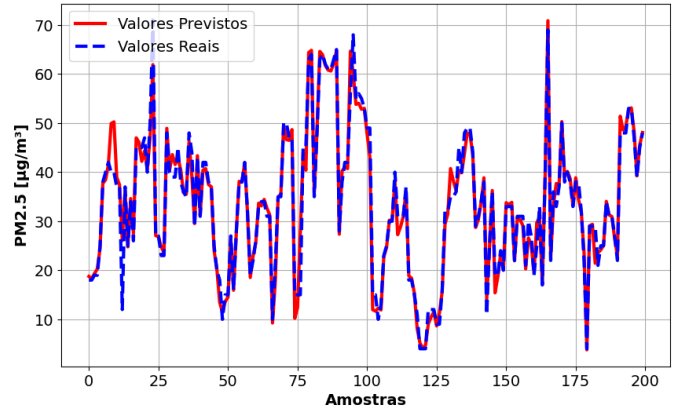
Observa-se que a curva de validação se estabiliza rapidamente, comportamento que poderia indicar o início de um processo de sobreajuste. Entretanto, graças ao mecanismo de parada antecipada, o treinamento foi interrompido antes que esse fenômeno pudesse comprometer a capacidade de generalização do modelo.

Como resultado da estimativa da concentração de PM_{2.5}, a Figura 14 apresenta a comparação entre os valores estimados pelo modelo e os valores reais para as primeiras 200 amostras do conjunto de teste.

Conforme pode ser observado na Figura 14, o modelo é capaz de fornecer estimativas bastante próximas dos valores reais. Para corroborar essa observação de forma quantitativa, foi calculada a métrica R^2 considerando todo o conjunto de teste, obtendo-se o valor de $R^2 = 0.944$.

Esse resultado é ligeiramente inferior ao melhor valor encontrado durante a busca aleatória. No entanto, essa diferença é esperada, uma vez que os resultados da busca foram obtidos

Figura 14: Estimativa de PM_{2.5} para as 200 primeiras amostras do conjunto de teste utilizando o melhor modelo encontrado na busca aleatória



Fonte: Autoria Própria

sobre o conjunto de validação, enquanto o valor reportado nesta etapa foi calculado utilizando o conjunto de teste, composto por amostras que não participaram nem do treinamento nem do processo de otimização dos hiperparâmetros.

Mesmo assim, o desempenho obtido supera o resultado apresentado no artigo de referência, que reportou um valor de $R^2 = 0.932$. Além disso, o modelo proposto alcançou esse desempenho utilizando uma arquitetura mais simples, composta por um menor número de camadas ocultas e um menor número de neurônios por camada. Adicionalmente, a taxa de aprendizado otimizada mostrou-se superior à utilizada na referência, favorecendo uma convergência mais rápida durante o treinamento.

A principal desvantagem observada em relação ao modelo de referência foi a adoção de uma janela temporal de 30 amostras, em comparação com as 20 amostras utilizadas originalmente. Entretanto, conforme discutido na seção VI-B, o tamanho da janela apresentou influência limitada sobre o desempenho final do modelo. Dessa forma, é razoável supor que a utilização de uma janela de 20 amostras, como na configuração original, não resultaria em alterações significativas no valor final de R^2 .

VII. CONCLUSÃO

Neste trabalho foi realizada a reprodução de uma rede neural do tipo MLP para a correção temporal de dados de concentração de PM_{2.5} obtidos por sensores de baixo custo, conforme proposto por [4]. A reprodução da metodologia mostrou-se bem-sucedida, alcançando resultados compatíveis com aqueles reportados no trabalho de referência.

Além da reprodução do modelo original, foi conduzido um processo sistemático de otimização de hiperparâmetros por meio de busca aleatória estruturada em duas etapas: exploração inicial e refinamento. Essa estratégia permitiu uma análise mais aprofundada da influência dos diferentes hiperparâmetros

sobre o desempenho da rede, fornecendo informações que não foram discutidas em detalhes no trabalho de referência. Adicionalmente, o processo adotado neste trabalho utilizou explicitamente conjuntos independentes de treinamento, validação e teste, contribuindo para uma avaliação mais robusta da capacidade de generalização do modelo.

A análise dos resultados demonstrou que a quantidade de neurônios por camada possui influência significativa sobre o desempenho da rede, enquanto o aumento da profundidade da arquitetura apresentou impacto limitado nos valores de R^2 . Também foi observado que a função de ativação ReLU apresentou maior robustez frente às variações da taxa de aprendizado, destacando-se como a alternativa mais adequada dentro do espaço de busca investigado.

Como principal resultado, o modelo otimizado alcançou um coeficiente de determinação de $R^2 = 0.944$ no conjunto de teste, superando o desempenho reportado no artigo de referência, que obteve $R^2 = 0.932$. Esse resultado foi alcançado utilizando uma arquitetura mais simples, composta por um menor número de camadas ocultas e um número menor de neurônios por camada, evidenciando que uma configuração otimizada pode proporcionar melhor desempenho sem a necessidade de aumentar a complexidade estrutural da rede.

Dessa forma, os resultados obtidos demonstram a eficácia da busca aleatória como método de otimização de hiperparâmetros e reforçam a importância desse processo no desenvolvimento de modelos de redes neurais artificiais. Além disso, o trabalho confirma o potencial das redes MLP para a correção temporal de medições de PM2.5 realizadas por sensores de baixo custo, contribuindo para a obtenção de estimativas mais precisas da qualidade do ar.

VIII. DISPONIBILIDADE DOS CÓDIGOS E DADOS

Os programas utilizados na execução deste trabalho, bem como os dados fornecidos pelos autores da referência utilizada, estão disponíveis em um repositório no GitHub acessível em [9].

REFERÊNCIAS

- [1] C. A. Pope III, N. Coleman, Z. A. Pond, and R. T. Burnett, "Fine particulate air pollution and human mortality: 25+ years of cohort studies," *Environmental research*, vol. 183, p. 108924, 2020.
- [2] United States Environmental Protection Agency, "Naaqs table," U.S. EPA, 2025, acesso em: 2 jun. 2026. [Online]. Available: <https://www.epa.gov/criteria-air-pollutants/naaqs-table>
- [3] R. Jayaratne, X. Liu, P. Thai, M. Dunbabin, and L. Morawska, "The influence of humidity on the performance of a low-cost air particle mass sensor and the effect of atmospheric fog," *Atmospheric Measurement Techniques*, vol. 11, no. 8, pp. 4883–4890, 2018.
- [4] M. Casari, L. Po, and L. Zini, "Airmpl: A multilayer perceptron neural network for temporal correction of pm2. 5 values in turin," *Sensors*, vol. 23, no. 23, p. 9446, 2023.
- [5] I. Goodfellow, "Deep learning," 2016.
- [6] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of machine learning research*, vol. 13, no. 2, 2012.
- [7] R. Rocha, "R quadrado," <https://medium.com/@lg0702/r-quadrado-3318259ff2e2>, 2021, acessado em: 07 jun. 2026.
- [8] M. Casari and L. P. Zini, "Airmpl—data," 2023, acessado em: 07 jun. 2026. [Online]. Available: <https://zenodo.org/records/10037781>

- [9] M. Farias, "Processamento adaptativo de sinais 2026.1," 2026, acessado em: 19 jun. 2026. [Online]. Available: https://github.com/matheusloop/Processamento_Adaptativo_de_Sinais_2026.1